

```
In [2]: # this line makes figures interactive in Jupyter notebooks
%matplotlib inline
from matplotlib import pyplot as plt

import numpy as np
from scipy.optimize import root_scalar
from scipy import constants
import cantera as ct

from pint import UnitRegistry
ureg = UnitRegistry()
Q_ = ureg.Quantity

# for convenience:
def to_si(quant):
    '''Converts a Pint Quantity to magnitude at base SI units.
    ...

    return quant.to_base_units().magnitude
```

```
In [ ]:
```

```
In [3]: def vertical_launch(t, y, spec_impulse, mass_prop, time_burn, gamma):
    '''Evaluates system of time derivatives for velocity, altitude, and mass.
    ...

    radius_earth = 6378.388 * 1e3
    gravity_ref = 9.80665
    density_ref = 1.225
    pressure_ref = 101325
    height_den = 10400
    height_pres = 8400

    v = y[0]
    h = y[1]
    m = y[2]

    gravity = gravity_ref * (radius_earth / (radius_earth + h))**2
    density = density_ref * np.exp(-h / height_den)
    pressure = pressure_ref * np.exp(-h / height_pres)

    mach = v / np.sqrt(gamma * pressure / density)
    #area = np.pi * diameter**2 / 4
    #drag = 0.5 * density * v**2 * area * coef_drag(mach)

    dmdt = -mass_prop / time_burn
    dvdt = (-spec_impulse * gravity_ref / m) * dmdt - gravity
    dhdt = v

    return [dvdt, dhdt, dmdt]
```

```
In [4]: def get_area_ratio(pressure_ratio, gamma):
    '''Calculates area ratio based on specific heat ratio and pressure ratio.
    pressure ratio: chamber / exit
    area ratio: exit / throat
```

```

'''
return (
    np.power(2 / (gamma + 1), 1/(gamma-1)) *
    np.power(pressure_ratio, 1 / gamma) *
    np.sqrt((gamma - 1) / (gamma + 1) /
            (1 - np.power(pressure_ratio, (1 - gamma)/gamma))
    )

def root_area_ratio(pressure_ratio, area_ratio, gamma):
    ''' Returns zero for a given area ratio, pressure ratio, and gamma.
    pressure ratio: chamber / exit
    area ratio: exit / throat
    '''
    return area_ratio - get_area_ratio(pressure_ratio, gamma)

def get_thrust_coeff(pressure_ratio, gamma):
    '''Calculates thrust coefficient for optimum expansion.
    pressure ratio: chamber / exit
    area ratio: exit / throat
    '''
    return np.sqrt(
        2 * np.power(gamma, 2) / (gamma - 1) *
        np.power(2 / (gamma + 1), (gamma + 1)/(gamma - 1)) *
        (1 - np.power(1.0 / pressure_ratio, (gamma - 1)/gamma))
    )

def get_cstar(Tc, MW, gamma):
    '''Calculates cstar using chamber properties.'''
    return (
        np.sqrt(Ru * Tc / (gamma * MW)) *
        (2 / (gamma + 1))**(-0.5*(gamma + 1)/(gamma - 1))
    )

```

```

In [5]: delta_velocity = Q_(2300, 'm/s')
        mpl = Q_(100000, 'kg')
        propellant_mass_fraction = 0.9
        burn_time = Q_(80, 's')
        Pc_initial = Q_(1000, 'psi')
        g0 = Q_(9.81, 'm/s^2')
        #For Sea Level, we need to choose a nozzle large enough.
        Pe = Q_(14.7, 'psi') #From https://www.noaa.gov/jetstream/atmosphere/air-pressure
        pressure_ratio = Pc_initial / Pe

```

```

In [6]: mixture_ratios = np.arange(2, 3.01, 0.1) #Mixture ratios tested
        with open('RP1_LOX_optimumOF.txt', 'r') as f:
            lines = f.readlines()

        c_star_test = []

        for line in lines:
            # ignore blank lines
            if not line.strip():
                continue

```

```

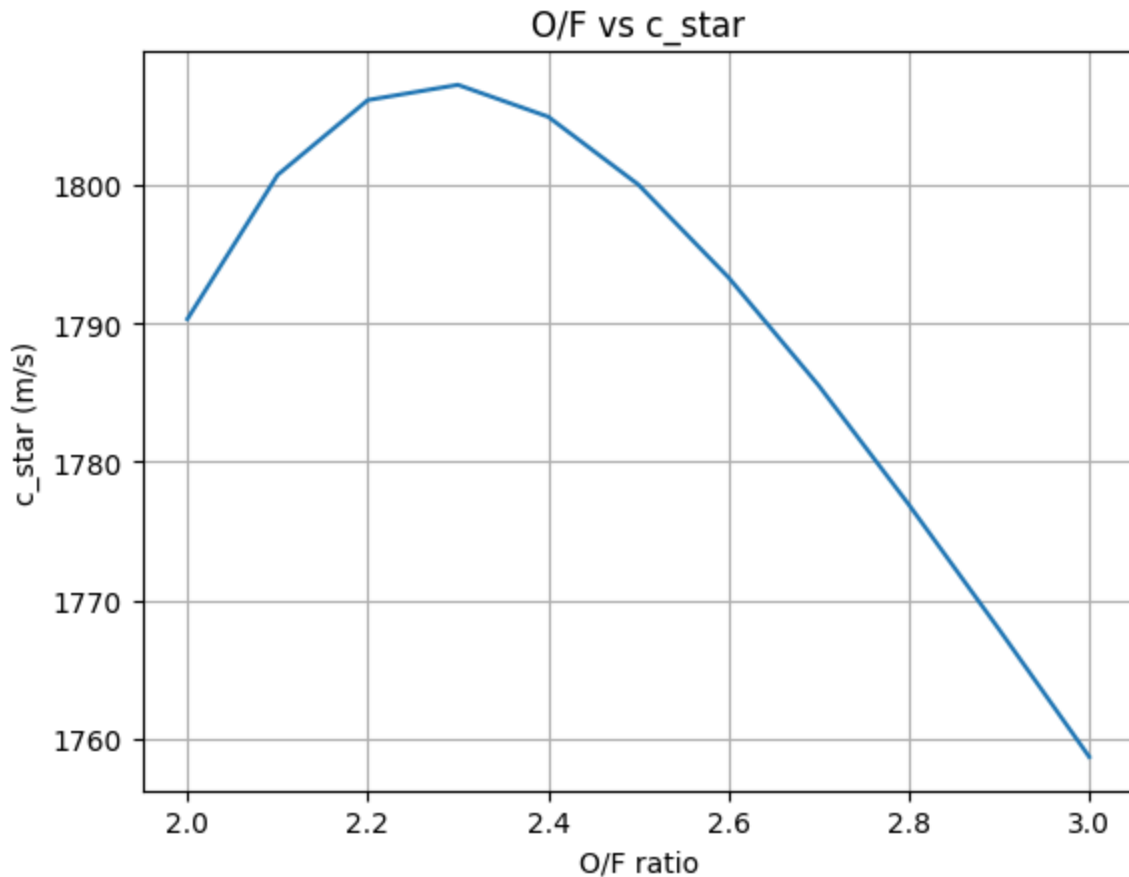
words = line.split()
if words[0] == 'CSTAR,':
    c_star_test.append(float(words[3]))

c_star_test = Q_(np.array(c_star_test), 'm/s')

plt.plot(mixture_ratios, c_star_test.magnitude)
plt.grid(True)
plt.xlabel('O/F ratio')
plt.ylabel('c_star (m/s)')
plt.title('O/F vs c_star')

```

Out[6]: Text(0.5, 1.0, 'O/F vs c_star')



```

In [7]: #Propellant Combination 1: RP-1 & LOX
#Using NASA CEA, with RP-1 and LOX, testing for the best c_star from 2-4 with inter
r = 2.3

#Using CEA again with this pressure ratio, we pull the values
gamma = 1.213
c_star = Q_(1807.2, 'm/s')
CF = get_thrust_coeff(pressure_ratio, gamma) #Optimum expansion assumption at seq_l
Isp = Q_((2917.9+2796.2) / 2, 'm/s') / g0 #To account for frozen and equilibrium
area_ratio = get_area_ratio(pressure_ratio, gamma)

fuel_density = Q_(811.86, 'kg/m^3') #Values Obtained from https://tsapps.nist.gov/p
oxidizer_density = Q_(1141, 'kg/m^3') #Value obtained from https://lasers.colostate

```

```
In [8]: print('The Optimum O/F is', r)
print('The Pressure Ratio is', f'{pressure_ratio.magnitude:.3}')
print('The CF is', f'{CF.magnitude:.3}')
print('The Isp at sea level is', f'{Isp:.4}')
print('The Area Ratio is', f'{area_ratio.magnitude:.3}')
```

The Optimum O/F is 2.3
The Pressure Ratio is 68.0
The CF is 1.59
The Isp at sea level is 291.2 second
The Area Ratio is 8.65

```
In [9]: #For the masses of propellants, we are going to use Tsiokvsky Rocket equation
mo_mf = np.exp(delta_velocity / (Isp * g0))
#Using the propellant mass fraction of 0.9, we can use systems of equations to solve
ms = mpl * (mo_mf - 1) / (10 - mo_mf)
mp = 9 * ms
fuel_mass = mp / (1 + r)
oxidizer_mass = fuel_mass * r

fuel_volume = fuel_mass / fuel_density
oxidizer_volume = oxidizer_mass / oxidizer_density
total_volume = fuel_volume + oxidizer_volume

#We are given the time burn, so we can find the mass flow rate
mdot_propellant_avg = mp / burn_time

#We also can find the Liftoff mass
liftoff_mass = mpl + ms + mp
#Requirements ask for twice the booster Liftoff mass for average thrust
thrust_requested = liftoff_mass * g0 * 2 #Is it Liftoff mass or is it the mass of t
#We can also use the values we have to calculate the area of throat required for th
#At_RP = avg_thrust_RP / (CF_RP * Pc_initial) #Maybe we cant use this one because t

#If we assume a constant mass flow rate, our thrust at any time is
Thrust_produced = Isp * g0 * mdot_propellant_avg

mdot_initial = Q_(2500, 'kg/s')

At = c_star * mdot_propellant_avg / Pc_initial
Rt = np.sqrt(At / np.pi)
Dt = 2*Rt

#Using the Area Ratio we found before, we can get the exit radius
Ae = At * area_ratio
Re = np.sqrt(Ae / np.pi)
De = 2 * Re

#Using the textbook, and applying the area_ratio and assuming a 15 degree conical n
nozzle_length_ratio = 7
nozzle_length = nozzle_length_ratio * Dt

#Lets assume a concentration ratio of 1.3
CR = 1.3

Ac = CR * At
```

```

Rc = np.sqrt(Ac / np.pi)
Dc = 2*Rc
#We also have from the notes, that the common L_star should be a minimum of 1, we w
#We can now find the expected length of our chamber using the
L_star = Q_(2, 'm')
L = L_star * (1 / CR)

#In this case, the fuel and oxidizer can be stored outside of the combustion chamber

```

```

In [10]: print('The starting mass to final mass ratio is', f'{mo_mf.magnitude:.3}')
print('The structural mass is', f'{ms:.6}')
print('The propellant mass is', f'{mp:.7}')
print('The mass of fuel is', f'{fuel_mass:.6}')
print('The mass of oxidizer is', f'{oxidizer_mass:.6}')
print('The volume of fuel is', f'{fuel_volume:.6}')
print('The volume of oxidizer is', f'{oxidizer_volume:.6}')
print('The total volume needed is', f'{total_volume:.6}')
print('The average mass flow rate of propellant is', f'{mdot_propellant_avg:.6}')
print('The liftoff mass is', f'{liftoff_mass:.7}')
print('The throat area is', f'{At.to('m^2')}')
print('The throat radius is', f'{Rt.to('m')}')
print('The throat diameter is', f'{Dt.to('m')}')
print('The exit area is', f'{Ae.to('m^2')}')
print('The exit radius is', f'{Re.to('m')}')
print('The exit diameter is', f'{De.to('m')}')
print('The nozzle length is', f'{nozzle_length.to('m')}')
print('The chamber area is', f'{Ac.to('m^2')}')
print('The chamber radius is', f'{Rc.to('m')}')
print('The chamber diameter is', f'{Dc.to('m')}')
print('The chamber length is', f'{L:.6}')
print('The requested thrust is', f'{thrust_requested.to('N'):.8}')
print('The Thrust produced is', Thrust_produced.to('N')) #The resulting thrust is L

```

The starting mass to final mass ratio is 2.24
 The structural mass is 15930.9 kilogram
 The propellant mass is 143378.1 kilogram
 The mass of fuel is 43447.9 kilogram
 The mass of oxidizer is 99930.2 kilogram
 The volume of fuel is 53.5165 meter ** 3
 The volume of oxidizer is 87.5813 meter ** 3
 The total volume needed is 141.098 meter ** 3
 The average mass flow rate of propellant is 1792.23 kilogram / second
 The liftoff mass is 259309.0 kilogram
 The throat area is 0.46976444984683496 meter ** 2
 The throat radius is 0.3866919556493736 meter
 The throat diameter is 0.7733839112987472 meter
 The exit area is 4.06162369843338 meter ** 2
 The exit radius is 1.1370378081531485 meter
 The exit diameter is 2.274075616306297 meter
 The nozzle length is 5.41368737909123 meter
 The chamber area is 0.6106937848008855 meter ** 2
 The chamber radius is 0.44089666491494156 meter
 The chamber diameter is 0.8817933298298831 meter
 The chamber length is 1.53846 meter
 The requested thrust is 5087643.1 newton
 The Thrust produced is 5120480.936598087 newton

In [16]: *#Assuming Tanks are held away from the chamber*

```

Df = Q_(3, 'm')

L_tank = total_volume / (np.pi * (Df/2)**2)
print('The length of storage tank is', L_tank)

L_fuel = fuel_volume / (np.pi * (Df/2)**2)
L_oxidizer = oxidizer_volume / (np.pi * (Df/2)**2)

print('The length of storage tank for fuel is', L_fuel)
print('The length of storage tank for oxidizer is', L_oxidizer)

```

The length of storage tank is 19.961250642552866 meter
 The length of storage tank for fuel is 7.5710379769266085 meter
 The length of storage tank for oxidizer is 12.39021266562626 meter

In [17]: **from** scipy.integrate **import** solve_ivp

```

sol = solve_ivp(
    vertical_launch, [0, burn_time.magnitude], [0, 0, liftoff_mass.to('kg').magnitude],
    args=(Isp.magnitude, mp.to('kg').magnitude, burn_time.magnitude, gamma),
    dense_output=True
)

```

In [18]: `time = np.linspace(0, burn_time.magnitude, 100)`
`z = sol.sol(time)`

```

acceleration = np.gradient(z[0, :], time)

fig, axes = plt.subplots(3, 1)

axes[1].plot(time, z[0, :])

```

```

axes[1].set_ylabel('Velocity (m/s)')
axes[1].grid(True)

axes[0].plot(time, z[1, :] / 1000)
axes[0].set_ylabel('Altitude (km)')
axes[0].grid(True)

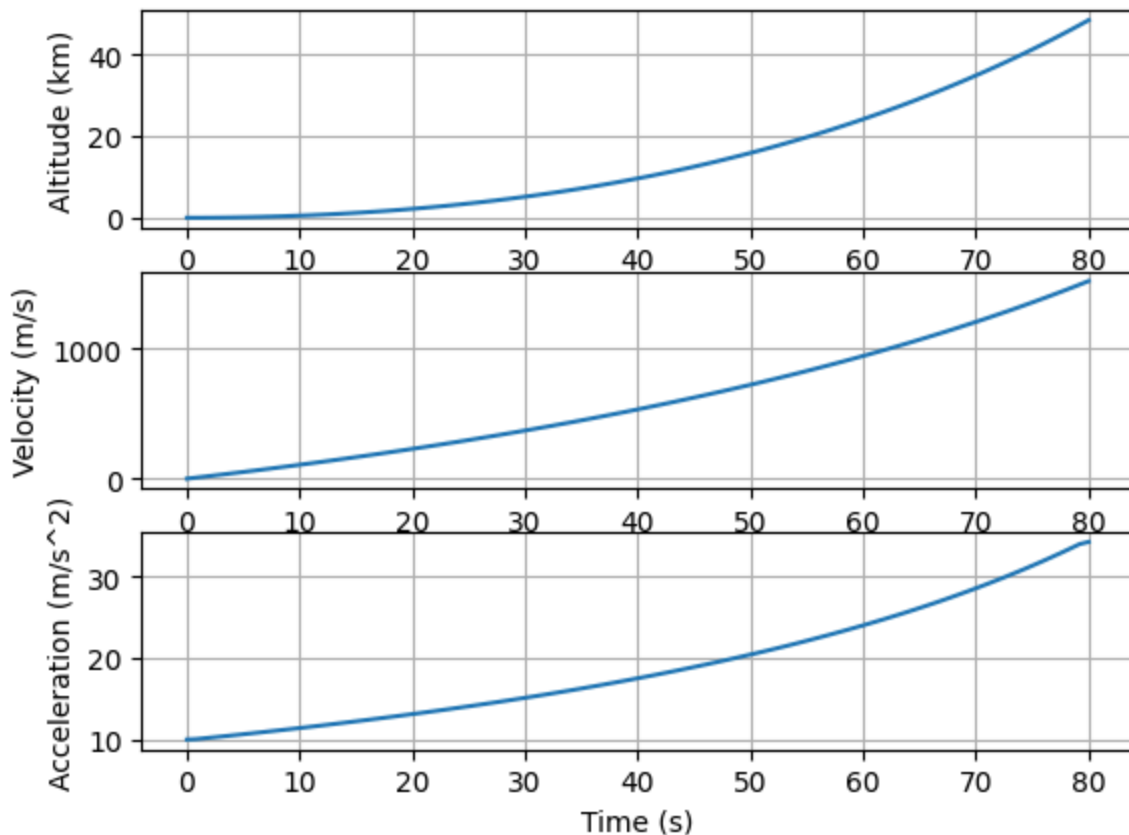
axes[2].plot(time, acceleration)
axes[2].set_xlabel('Time (s)')
axes[2].set_ylabel('Acceleration (m/s^2)')
axes[2].grid(True)

plt.suptitle('Rocket Performance for RP-1 & LOX', fontsize=16, fontweight='bold')

plt.show()

```

Rocket Performance for RP-1 & LOX



In [19]: *#Note for self - Maybe try using the requested thrust to find what the ideal Isp is*

```

In [23]: acceleration_initial = Q_(10, 'm/s^2') #From graphs, Looks Like the acceleration ri
max_pressure_fuel = fuel_density*(g0+acceleration_initial)*L_fuel
max_pressure_oxidizer = oxidizer_density*(g0+acceleration_initial)*L_oxidizer
yield_strength = Q_(310, 'MPa') #https://asm.matweb.com/search/specifcmaterial.asp
tensile_strength = Q_(276, 'MPa')
Tw = min((yield_strength)/1.1, (tensile_strength)/1.25) #Assuming uncrewed

t_c_fuel = max_pressure_fuel * (Df/2) / Tw
t_c_oxidizer = max_pressure_oxidizer * (Df/2) / Tw

```

```
#print(max_pressure.to('Pa'))  
print('Maximum allowable working stress', Tw)  
print('Thickness for fuel tank', t_c_fuel.to('mm'))  
print('Thickness for oxidizer tank', t_c_oxidizer.to('mm'))
```

Maximum allowable working stress 220.8 megapascal
Thickness for fuel tank 0.8272051595752901 millimeter
Thickness for oxidizer tank 1.9025718670231668 millimeter

In []: